

Stock Management Mobile App

Architecture Strategy & Roadmap without Android Studio

Note: Not having Android Studio installed is not a dealbreaker for building or deploying a production-ready mobile application. You can develop, debug, and build full-fledged mobile applications using lightweight code editors (like VS Code) and cloud-based compilation services.

1. Frontend Alternatives (No Local Android Studio)

Option A: React Native + Expo RECOMMENDED

Since your backend API will be written in **TypeScript (Node.js)**, selecting React Native allows you to maintain a unified language stack across both frontend and backend.

- **Development Workflow:** Expo allows developing mobile apps without heavy native IDEs or local Android emulators.
- **Real-time Testing:** Run `npx expo start` in VS Code, scan the generated QR code using the **Expo Go** mobile app, and stream code updates live to your physical phone.
- **Cloud Builds (APKs):** Use **EAS Build** (Expo Application Services) in the cloud. Remote servers compile your `.apk` / `.aab` binary automatically.

Option B: Flutter via VS Code + Web / Physical Device

If you prefer using Dart/Flutter, you can configure your workspace entirely within VS Code without installing Android Studio:

- **VS Code Setup:** Install the official **Flutter & Dart extensions** directly inside VS Code.
- **Testing without Emulator:** Debug and run Flutter web apps in **Chrome** or connect an Android mobile device via USB with **Developer Mode / ADB enabled**.
- **Cloud Builds:** Services like **Codemagic** or **GitHub Actions** can compile your production-ready `.apk` binaries via cloud CI/CD pipelines.

2. Recommended Tech Architecture

LAYER	TECHNOLOGY	ROLE IN INVENTORY SYSTEM
Frontend	React Native (Expo) or Flutter	Role-based UI interfaces (Owner/Manager vs. Staff views).
Database & Auth	Supabase (PostgreSQL)	Stores inventory items, user roles, transaction logs, and handles real-time sync.
Backend API	Node.js / Express (TypeScript)	Custom microservices (purchase orders, batch imports/exports, complex data processing).

LAYER	TECHNOLOGY	ROLE IN INVENTORY SYSTEM
Real-time Engine	WebSockets / Supabase Realtime	Pushes live stock updates immediately when stock is checked in or out.
Push Notifications	Firebase (FCM) / Expo Notifications	Low-stock threshold alerts pushed directly to administrative mobile devices.
Hosting & CI/CD	Render / Supabase Cloud / Railway	Cloud hosting for Node.js services and managed PostgreSQL database instance.

3. Step-by-Step Development & Launch Roadmap

Phase 1: Database & Schema Architecture

- **Database Schema Setup:**
 - `users` (id, role: `owner` | `manager` , name, email)
 - `products` (id, sku, name, total_stock, minimum_threshold)
 - `stock_transactions` (id, product_id, change_type: `IN` | `OUT` , quantity, created_by, timestamp)
- **Security & Rules:** Configure Row-Level Security (RLS) policies so that only authorized users can modify inventory quantities.
- **Real-time Activation:** Enable Supabase Realtime tracking on the `products` and `stock_transactions` tables.

Phase 2: Microservices & Logic Layer

- **Express/Node.js REST Service:** Implement endpoints for bulk transactions, audit trail generation, and report exports.
- **Notification Trigger Engine:** Set up webhooks or automated triggers:
 - When `total_stock <= minimum_threshold` , issue automated push notifications via FCM or Expo Notifications to admins.

Phase 3: Frontend Interface Development

- **Initialize Workspace:** Run `npx create-expo-app stock-manager` in VS Code.
- **Key Screens:**
 - **Dashboard:** Real-time analytical graphs tracking incoming and outgoing stock trends.
 - **Inventory Scanner:** Integrated QR / barcode scanning using `expo-camera` .
 - **Stock Action Screen:** Rapid update interface for quick stock additions and removals.
- **Socket Subscription:** Subscribe to database real-time channels so UI components auto-refresh without manual pulls.

Phase 4: Cloud Build & Production Launch

- **Backend API Deployment:** Containerize or push your Node.js application directly to **Render** or **Railway**.
- **Mobile Application Compilation:** Execute `eas build -p android --profile preview` using Expo Application Services to build and download the final `.apk` installer for direct deployment onto target devices.